Feedback scheduling and a source control obstruction

Adjacent refutation codes and exact source verification

21 October 2026

1 Status and scope

This report does not establish a new calibrated 215-state machine or a 214-state machine. It records a useful completeness theorem for a state-dependent scheduler and a concrete source-control obstruction found while trying to apply it to a published 215-state experiment. The underlying physical table is inherited, not newly invented here.

The positive mathematical result is an adjacent-pair construction: every universal refutation word has cofinally many pairs of consecutive candidate codes that both halt. Any increasing candidate sequence whose gaps are one or two must therefore encounter a halting candidate. This replaces an unjustified fairness assumption for a feedback-dependent scheduler by an explicit arithmetic proof.

The investigated source has 201 DAG nodes, 4,733 primitive occurrences and 6,152 guarded primitive cases. A fresh universal runtime replay proves all 6,152 cases on the exact 215-state binary table. Nevertheless, two independent source-control audits find that its altered continuation returns to the selector, not to the scheduler. Thus the physical runtime theorem does not establish the advertised high-level command loop. In fact, the actual first AX completion is followed by an unconditional halt. The research-only source is rejected; this does not affect the separately calibrated MP-first transfer construction.

The investigation is pinned to ProveIt main commit

`f5b2db607f8cf1daccf601b4ce14d7cee38b6770.`

The physical experiment was already present in the short-transfer research component of `T215_Transfer_2026-10-20.zip`. Its upstream README explicitly left logical calibration open. The current contribution is the conditional scheduler theorem, the corrected 217-state prototype, recognition that the reported startup stack is entirely theorem-valued, and the independently reproduced return-target error. No Lean or Rocq endpoint for this table is claimed.

2 The numerical calculus and command interface

Write

$$P(a,b) = \frac{(a+b)(a+b+1)}{2} + b, \qquad I(a,b) = 4P(a,b), \qquad Q_i a = 4P(i,a) + 2.$$

Truth is zero, falsity is one, and $[x;w]$ abbreviates $P(x,w)$. These are operations on every natural number. In particular $I(0,0) = 0$ and $P(0,0) = 0$ are intentional aliases, not exceptions to be silently excluded.

The inherited dominated numerical calculus retains truth, numerical MP, theorem-only generalization, seven logical rows, the full original gated set-theoretic bundle and the dominated classical schema

$$L(A,B,C,U) = I\big(I(I(C,U),A), I(I(A,C), I(B,C))\big), \qquad U \le A.$$

The repository's calibration identifies derivability of the raw code one with existence of a finite ZFC refutation. The supplied inherited finite certificate checks the recovery of the unrestricted classical schema over all natural substitutions. Its replay has 3,728 proof nodes and 5,160 term-table nodes. This finite propositional certificate is one identified dependency of the calibration, not by itself a proof of the whole set-theoretic interpretation.

A work store is (t, w, d, h): top formula, paired stack, additive parameter and cached formula. The short transfer command has the exact intended interface

$$S(t, [x; v], d, h) = \begin{cases} \text{halt}, & h = 1, \\ (h, v, d, x), & h \neq 1. \end{cases}$$

It tests only the incoming cache, unlike the composite transfer $D; N; N$, which also tests the old top.

For $\mathsf{MP}(x)$ put $y = d + x$. Its intended interface is

$$\mathsf{MP}(x)(t, w, d, h) = \begin{cases} (y, [0; w], y, 0), & I(t, y) = h, \\ (0, w, y, 0), & I(t, y) \neq h. \end{cases}$$

For $\mathsf{AX}(c; x, y)$ put $U = d$, $A = d + x$ and $B = h + y$. It produces the full bundle at (A, B, c) when $c \neq A, B$, and zero otherwise, while pushing

$$r_1, \ldots, r_7, L(A, B, c, U), Q_U h, t$$

above the old stack and resetting $d = h = 0$. Every first-nine entry is nonfalsity: the rows and L have implication tag zero, while $Q_U h$ has tag two. The saved top is the tenth entry. The field lists are

$$\text{fields}(S) = [0], \quad \text{fields}(\mathsf{MP}(x)) = [1, x], \quad \text{fields}(\mathsf{AX}(c; x, y)) = [c + 2, y, x].$$

3 The inherited strong refutation compiler

The direct short-transfer proof is included as `proofs/short_transfer_compiler.md`. It is important to use that direct proof, rather than infer completeness by removing a halt test from a different implementation.

Let $D = \mathsf{AX}(0; 0, 0)$ and define

$$H = D; S^{11}, \qquad \mathsf{PUSH} = D; S^9; \mathsf{MP}(0).$$

From $(t, [x; w], d, h)$, H halts exactly when $t = 1$ and otherwise returns $(t, w, 0, x)$. The ten AX entries allow the original top to enter the cache just before the eleventh transfer. By contrast, PUSH never halts and returns $(0, [t; w], 0, 0)$. Its final MP necessarily fails because the cache has quantifier tag two and an implication has tag zero.

Define

$$\mathsf{DISCARD} = \mathsf{MP}(1); \mathsf{AX}(1; 0, 0); S^{10}; \mathsf{MP}(0).$$

On every $(t, w, 0, 0)$ it returns $(0, w, 0, 0)$. At top, parameter and cache zero, set $J(0)$ to the empty word and $J(u) = \mathsf{MP}(u)$ for $u > 0$. Then $J(u)$ sets the parameter to u without changing the stack. The empty zero case matters: otherwise the raw alias $I(0, 0) = 0$ would make MP succeed and push an unwanted zero.

The compiler proves by induction on finite derivations that every theorem X has a block B_X satisfying

$$B_X(t, w, 0, 0) \quad \text{halts or returns exactly } (X, w, 0, 0)$$

for *all* natural t, w. Selected rows, the full bundle and generalization are recovered by the displayed padding macros. For MP with premises A and $I(A, B)$ the block is

$$B_{I(A,B)}; \mathsf{PUSH}; B_A; \mathsf{PUSH}; S; S; \mathsf{MP}(B); \mathsf{AX}(B; 0, 0); S^{11}.$$

The two transfers arrange A on top and $I(A, B)$ in cache. Successful MP pushes a zero; the last AX and eleven transfers remove precisely that zero and restore the required normalized store, or halt when $B = 1$. The proof handles $A = B = 0$ without assuming injective list length.

Consequently, if the calculus proves one, $B_1; H$ halts from every $(t, w, 0, 0)$. The word

$$W = \mathsf{AX}(0; 0, 0); B_1; H$$

halts from every natural work store: the initial AX resets parameter and cache, and the strong compiler allows its arbitrary resulting top and stack. This universality is what allows the two different candidate prefixes below to converge only in control, not in data.

4 An abstract feedback scheduler

The following theorem concerns an *explicitly defined abstract scheduler*. The archived physical table has not been shown to implement it; Section 9 explains why.

Let $b(S) = 0$ and $b(\mathsf{MP}) = b(\mathsf{AX}) = 1$. At a positive residual cursor L, read the first command field by unpairing $L - 1 + b$, clear the bias, and read any remaining fields by ordinary unpairing. The bias for the next boundary is determined by the command just executed. At an empty cursor, a refill enters the next candidate and increases the candidate counter by either one or two. The abstract contract requires the first field of a newly entered candidate n to be read directly from n.

Lemma 1 (Finite draining)**.** *Every nonhalting candidate finishes in finitely many commands.*

Proof. Let $f(n)$ be the second projection of Cantor unpairing. For $n > 0$, $f(n) < n$. Indeed $P(a, y) > y$ whenever $P(a, y) > 0$: this is immediate for $a \geq 1$, and for $a = 0$, $P(0, y) = y(y + 3)/2 > y$ when $y > 0$.

At a positive boundary, the new residual is one, two or three right projections of $L - 1 + b$. If $b = 0$, it is at most $L - 1$. If $b = 1$, its first projection is $f(L) < L$ and the later projections cannot increase it. Every command has finite arithmetic duration under its intended contract. Thus the residual strictly descends until refill, unless the machine halts. □

The candidate values in a nonhalting execution therefore form an unbounded increasing sequence with gaps in $\{1, 2\}$. The gap convention must be checked against a literal implementation: whether the increment uses the preceding command's bias is an indexing detail that must not be confused with the candidate currently being entered. The arithmetic theorem below uses only the gap bound and the specified candidate-entry decoding.

5 Encoding a word under feedback

For a list of fields $a_1, \ldots, a_k$, write

$$F([a_1, \ldots, a_k], r) = P(a_1, P(a_2, \ldots, P(a_k, r) \ldots)).$$

Fix a nonempty command word $W = (C_0, \ldots, C_{m-1})$ and a positive unused terminal residual $r \geq 1$. Put $R_m = r$ and, backwards for $i = m - 1, \ldots, 1$, set

$$R_i = 1 - b(C_{i-1}) + F(\text{fields}(C_i), R_{i+1}).$$

Define $K = F(\text{fields}(C_0), R_1)$, with the evident convention $K = F(\text{fields}(C_0), r)$ for $m = 1$.

All R_i and K are positive because $P(a, z) \geq z$ and $r \geq 1$. When the next first-field read is K, the scheduler executes exactly W unless it halts earlier. At each later boundary, subtracting one and adding the old bias cancel the term $1 - b(C_{i-1})$ in R_i.

The positive terminal residual is essential. With terminal zero, an encoded trailing S following bias one could have residual zero; the scheduler would refill instead of executing it. Here the residual is never reached when W is universally halting, but its positivity still guarantees that all required preceding boundaries exist.

6 Adjacent universally halting candidates

Theorem 2 (Adjacent pair completeness)**.** *Suppose the command system has a finite word W that halts from every work store. For the abstract feedback scheduler there are cofinally many pairs of consecutive candidate numbers such that both members halt. Hence every nonhalting increasing candidate schedule with gaps at most two contradicts the existence of W.*

Proof. Choose a positive-residual encoding K of W as above. For each $x \geq 0$ set

$$z = 1 + K, \qquad n_x = P(1, P(x, P(0, z))).$$

Candidate n_x first executes $\mathsf{MP}(x)$, leaving residual $P(0, z)$ and bias one. Its next first-field read is $P(0, z) - 1 + 1 = P(0, z)$, so it executes S and leaves residual z with bias zero. The next read is $z - 1 = K$, giving W. Any earlier halt already establishes success.

For $b > 0$ the exact Cantor identity

$$P(a, b) - 1 = P(a + 1, b - 1)$$

holds because the diagonal index $a + b$ is unchanged. Three applications give

$$n_x - 1 = P(2, P(x + 1, P(1, K))).$$

All required right arguments are positive. This candidate begins with $\mathsf{AX}(0; 1, x + 1)$, leaving residual K and bias one. Its next read is $K - 1 + 1 = K$, so it also executes W. The two prefixes need not leave the same work store; universality of W handles both.

As x grows the pair $\{n_x - 1, n_x\}$ is unbounded. An increasing unbounded integer sequence with gaps at most two meets each sufficiently late adjacent pair. At its first value at least $n_x - 1$, the previous value is at most $n_x - 2$, so the new value is at most n_x. It is therefore one of the two halting candidates. □

This argument does not assert that an arbitrary cofinal set intersects an arbitrary subsequence. It uses a specific bounded-gap hitting set. It also does not require a fixed parity, arithmetic progression, equidistribution statement or probabilistic fairness assumption.

7 A uniform resource bound for bounded-gap schedules

A stronger optional theorem handles an opaque numerical tail rather than the entirely safe stack three. It concerns the same abstract feedback scheduler, not the rejected physical source. Assign conservative balances $S = -1$, $\mathsf{MP} = 0$, $\mathsf{AX} = 10$. The actual successful-MP zero push can only improve this lower bound.

Let $W(n)$ be the complete syntactic word decoded from candidate n, ignoring intended halts; let $s(n)$ count its S commands, and put $g(0) = 0$, $g(n) = s(n)$ for $n > 0$. With f the right Cantor projection, exact decoding yields

$$\begin{aligned} W(P(0, y)) &= S; W(y - 1) && (y > 0), \\ W(P(1, y)) &= \mathsf{MP}; W(f(y)) && (f(y) > 0), \\ W(P(a, y)) &= \mathsf{AX}; W(f^2(y)) && (a \geq 2,\ f^2(y) > 0). \end{aligned}$$

An indicated suffix with zero residual is omitted; $W(0) = S$, so $s(0) = 1$ while $g(0) = 0$.

Lemma 3 (Global transfer count)**.** *For every $N \geq 0$, $A(N) = \sum_{n<N} s(n) \leq N$.*

Proof. Use strong induction. Let $T_q = q(q+1)/2$. If a nonnegative function u vanishes at zero and has positive-length prefix sums at most $L-1$, its pullback by f has prefix sums at most $L - r(L)$, where $r(L)$ is the number of nonempty Cantor diagonal blocks before L, equivalently the least r with $T_r \geq L$. Indeed the sequence of right projections is

$$0; \quad 0,1; \quad 0,1,2; \quad \ldots,$$

and each complete or nonempty partial block loses at least one against its length. Induction gives $\sum_{j<L} g(j) = A(L) - 1 \leq L - 1$, so this applies to g and then again to $g \circ f$. Write their pullback sums as $H(L)$ and $H_2(L)$; each is at most $L - r(L)$.

For $q \geq 2$, decomposition by root command gives the exact identity

$$A(T_q) = q + A(q-1) + H(q-1) + \sum_{\ell=1}^{q-2} H_2(\ell).$$

It follows that

$$A(T_q) \leq T_q - \left(\sum_{\ell=1}^{q-1} r(\ell) - (q-1) \right) \leq T_q.$$

For $N = T_q + k$ with $0 \leq k \leq q-1$, the extra partial diagonal contains only AX roots and contributes $H_2(k) \leq k$. For $k = q$, its extra cost is $H_2(q-1) + g(f(q-1))$. The induction bounds $g(m) \leq m$ and $f(q-1) \leq r(q-1)$ make this at most $q-1$. All invoked indices are smaller than the induction target; $q = 0, 1$ are checked directly. □

Theorem 4 (Three initial heads suffice)**.** *For every increasing candidate sequence starting at one with gaps in $\{1,2\}$, every internal command prefix has conservative stack balance at least -3. Thus three initially safe heads above an arbitrary opaque tail suffice.*

Proof. Before candidate n there are at least $\lfloor (n-1)/2 \rfloor$ completed candidates. Let q be the Cantor diagonal index of n; then $q \leq \sqrt{2n}$. Among all indices through n, at most $2q+1$ have a non-AX first command. Consequently at least $\lfloor (n-1)/2 \rfloor - 2q - 1$ earlier selected candidates have already contributed an initial AX. No credit is borrowed from the current candidate.

At any point within candidate n, all transfer costs encountered are at most $A(n+1) \leq n+1$. Ignoring all further AX credits and successful-MP pushes gives the lower bound

$$10 \left\lfloor \frac{n-1}{2} \right\rfloor - 20q - 10 - (n+1) \geq 4n - 20\sqrt{2n} - 21.$$

This is positive at $n = 64$ and increases thereafter. An exact finite min-plus dynamic program checks every possible sequence with gaps one or two through candidates 1–63, including every internal command prefix. Its minimum is -3, attained along the permitted prefix $1, 2, 4, 5$. An independently implemented recursive decoder and dynamic program reproduce every one of the 63 records. The analytic estimate, not an empirical extrapolation, justifies the finite remaining domain. □

The resource invariant is an explicit finite wrapper of theorem heads over the opaque tail. Its length is not inferred as a numerical function of the paired stack, so zero-pairing collapse causes no ambiguity. The hypothesis that the sequence starts at one is explicit; this theorem is not silently applied to arbitrary later initial counters.

8 The reported startup stack is already safe

The exact blank run of the archived 215-state table reaches dispatcher 84 after 69,744 transitions, with head -2, source ordinal 64 and registers

$$(N, T, W, A, D, H, X, Y, L) = (1, 4, 0, 0, 0, 0, 3, 10, 0).$$

The source address is `10010010010110110110`. There are 65 spare blocks: the second is one and every other spare is zero. An independent 172-step source trace from AX entry ordinal 25 starts at

$$(N, T, W, A, D, H, X, Y, L) = (1, 0, 3, 0, 0, 0, 0, 0, 0)$$

and reaches exactly the displayed ordinal and all nine registers. Sparse-native, dense-native and canonical simulations check the entire finite support and blank exterior, not only a local tape window.

The initial stack is not an unexplained unsafe tail:

$$3 = P(2, 0), \qquad 2 = Q_0(0).$$

Truth zero is a theorem, so theorem-only generalization gives $Q_0(0) = 2$. Iterated unpairing of the remaining zero produces zero heads forever. Formally, call a stack safe if every entry of its infinite iterated head-projection sequence is a theorem. Zero is safe; pairing any theorem onto a safe stack preserves safety; therefore three is safe.

This observation removes a protected-credit obligation *for a correct implementation of the intended command system.* Safe top, cache and decoded stack heads are preserved by S, MP and AX, with unrestricted numerical parameter d. A halt of the intended S implies that the theorem one is present in cache. This argument does not repair the incorrect literal return discussed next.

9 The literal return target obstruction

The archived scratch-biased source inserts the pair $(\mathrm{CI6}, \mathrm{CC3})$ at the outer command return. The intended effect was to set the parser bias to one and return to the scheduler. But inserting a new right child changes the continuation's ancestor stack.

The exact leaf at address `100111` is CC3, source ordinal 4,697. Its third enclosing right ancestor is the selector at ordinal 23, not the scheduler root at ordinal zero. Both a structural interval-based resolver and an address-based resolver confirm this target. The specific claim "sets bias one and returns to scheduler" is therefore false for the actual source.

The error is not cosmetic. At source ordinal 4,696 suppose $A = H = X = Y = 0$, with arbitrary top $T = t$ and arbitrary other registers. The inserted CI6 sets $X = 1$, and the erroneous CC3 reaches the selector CD3 at ordinal 23. Its zero branch enters the clear-top loop at ordinal 4,698. This loop executes t decrements with two primitive steps per positive iteration, then exits with top zero. The cache-copy loop sees $H = 0$ and performs no copying, but its restoration loop consumes the unexpected scratch value $X = 1$ and increments H to one. The inherited NIP comparison then reaches CH at ordinal 4,718.

Hence the number of executed non-CH source instructions is exactly

$$2t + 15.$$

The owned check records the fifteen fixed instructions for $t = 0$ and the exact two-step natural-number loop. Additional finite probes corroborate the invariant; they are not the basis of its universal quantification.

For the actual blank startup, the entire AX body is the unchanged inherited terminating body. The source bridge starts with $U = A = B = C = 0$, so the gate is disabled and its final top is zero. Its

exit has $A = H = X = Y = 0$ and reaches ordinal 4,696. The source therefore reaches its halting leaf after fifteen more non-CH primitive instructions following the finite AX completion. Combined with the complete binary-runtime proof and whole-tape blank bridge, this establishes unconditional halting of this particular binary machine. No numerical consistency hypothesis or immense literal simulation of the intervening AX arithmetic is needed. The safe initial stack does not rescue the machine: the wrongly routed scratch bit creates cache one without a logical inference.

A local change from CC3 to another continuation, or a new source association, must satisfy both the intended source CFG and the contextual physical helper constraints. Merely restoring the abstract target does not by itself preserve a 215-state physical runtime. That repair remains a separate construction obligation.

10 What the exact physical replay does prove

The native table SHA-256 is

`f08bed9e62b9f43237b65c465d2f1bfc17474637b4b02785ca7f1813827bb0ce.`

The source SHA-256 is

`2407f815551b4aba335c9bf041e15e8ce3cfb9ad830d89539f3bcf123462371a.`

The explicit canonical table has start zero, working states $0, \ldots, 214$, and separately counted halt label 215. Its SHA-256 is

`0939bb4bba38bda4cd08d3201fd1cf1a6b87513cd5663a255cadd3e723a2b8d8.`

All 215 working states are graph reachable. The fresh whole-tape runtime replay proves 6,152 guarded primitive cases, uses 1,617,669 symbolic construction rules, covers all 430 transition rows and has minimum duration constant six. It quantifies over arbitrary natural primary registers, arbitrary gap and arbitrary nonempty finite lists of natural spare values. Increment appends spare value zero. The symbolic checker uses literal transitions, unary induction and independently rebuilt finite-list schemas.

These are genuine universal primitive contracts, not finite random tests. Their correctness is compatible with the source being the wrong program. They prove that the physical machine realizes its literal source operations, including the literal continuation target; they do not prove an informal intended macro contract contradicted by that source. This separation is the main verification lesson of the example.

11 A 200-node source with shorter zero runs

The latest inspected encoding packet already supplied an equivalent 200-node source with maximum zero run eight. A new owned closed-reassociation search reduces the maximum zero run to seven while retaining 200 distinct source DAG nodes and all 4,775 primitive occurrences. Its maximum address depth is 46. Exact comparison checks both the entire primitive word and every guarded ordinal CFG edge against the earlier calibrated transfer source.

The source identifier is

`d70f4004ab7d579e8ce06f545bbe18654201cc3a7f3df10adc3a9e2f500461b0.`

The 200 nodes comprise 177 distinct internal ordered pairs and 23 distinct primitive symbols. These counts are structural source facts, not a machine-state construction. The source still exceeds the inherited six-zero scanner constraint. More importantly, a row-protected physical allocator

produces 222 states: shorter source addresses do not automatically retain all earlier contextual helper sharing. The retained bounded search inspected 3,000 popped search nodes and 509,849 distinct proposals. Its failure to find a six-zero 200-node source is not an impossibility theorem.

12 A different command interface with stack-output MP

A further complete command-level experiment changes the location of an MP result. Put $y = d + x$ and let

$$M(x)(t, w, d, h) = \begin{cases} (0, [y; w], y, 0), & I(t, y) = h, \\ (0, [0; w], y, 0), & I(t, y) \neq h. \end{cases}$$

Every MP now pushes a head, including failed MP; successful results reside on the stack rather than on top. Retain the composite transfer $D; N; N$ and the unchanged AX. This is a semantic interface change, not a source relayout. Successful M pushes a theorem consequence and failed M pushes truth, so soundness is preserved at safe work boundaries.

Let $A_0 = \mathsf{AX}(0; 0, 0)$. The repaired structural blocks are

$$\begin{aligned} H &= A_0; S^{11}, \\ \mathsf{PUSH} &= A_0; S^9; M(0); S, \\ \mathsf{DISCARD} &= M(1); \mathsf{AX}(1; 0, 0); S^{10}; M(0); S^2, \\ J(u) &= M(u); S \qquad (u \geq 0). \end{aligned}$$

They have the same strong contracts required by the preceding compiler. In particular, $J(0)$ is now valid without a special empty-word convention: whether the zero-alias MP succeeds or fails, it pushes zero and the following transfer removes precisely that zero.

For a logical MP with major $K = I(A, B)$, use

$$\begin{gathered} \mathsf{DISCARD}; \mathsf{PUSH}; \mathsf{PUSH}; B_K; \mathsf{PUSH}; B_A; \mathsf{PUSH}; S; S; \\ M(B); S; S; \mathsf{AX}(B; 0, 0); S^{11}. \end{gathered}$$

The first blocks reserve two zero heads over the arbitrary suffix. The premise blocks and two transfers arrange $(A, [0, 0; w], 0, K)$. Successful M gives $(0, [B, 0, 0; w], B, 0)$, and the next two transfers give $(B, [0; w], B, 0)$ or halt when $B = 1$. The old cleanup then restores $(B, w, 0, 0)$. The finite proof induction and arbitrary-stack contract are complete; the full details and a drained-counter variant are included in the proof artifact.

The non-draining interface prices at 202 source nodes and 217 raw allocated states across both selector orders and both measured associations. A drained-counter success interface has a shorter MP proof block but prices at 203 nodes and 218 raw states. Neither raw allocation has a completed independent binary runtime or blank initialization proof, and neither is a calibrated 217- or 218-state construction. They are exact pricing evidence attached to a proved abstract compiler, not smaller machines.

13 Qualified physical-search results

Five exact finite-window context certificates were checked independently. Two-branch contextual alias screens covered four 215-state baselines and the provisional global-bias 216-state allocation. Each 215-state screen considered 45,796 ordered state pairs and about 39.5 million candidate branch actions; the 216-state screen considered 46,225 pairs and 40,063,100 branch actions. No survivor was found. A separate common-branch screen on the latter table examined 508 ordered pairs and 436,880 actions, also without a survivor.

These are deliberately bounded families, not impossibility results. The two-branch screen requires exact one-step expansion within 64 transitions, complete finite-patch restoration and no interior

recurrence to the merged state. The common-branch screen permits recurrence and uses a 128-transition bound. The packet contains the exact tables, contexts, inductive context checker, owned C++ sources and logs. Its default verifier rechecks the contexts, not the expensive synthesis runs. Separate 860-action local screens of the erroneous return used representative zero-register/unit-spare fixtures; their negative outcomes are not universal runtime proofs.

A separate corrected-return association family priced 80 exact-CFG sources under three provisional continuation actions, for 240 trials. It produced 24 raw 215-state allocations, but all fail a concrete valid first-instruction frame: CD8 at address 0000, nine zero registers, gap zero and spare list [0]. They redispatch after 152 steps with unwanted ones in cells 5–63 instead of the required PC bit at cell 2. The broader bounded source search examined 23,327 proposals and 14,489 valid unique CFG-preserving sources; its best metric was 202 nodes, depth 43 and maximum zero run six. These archived receipts are separately marked as not rerun by the package's default verifier.

14 Constant-bias prototype: a separate 217-state repair

A different source repair places one common CI6 footer after *every* command, followed by a literal root return. Thus the parser bias is always one, the first-field read at positive residual L unpairs L itself, and empty-cursor refill advances the candidate counter by two. The selected candidates are $1, 3, 5, \ldots$. This source has 4,734 primitive leaves. Its first raw 216-state allocation failed the universal primitive audit at the short-transfer top-clear instruction; it must not be advertised as a working 216-state machine.

An explicit one-state physical repair splits that clear node: native row 313/0 becomes 0R314, while new state 314 has rows 0R41 and 0L1. Native start is 74, not the allocator's generated header zero. The companion table and canonical relabeling are included separately. The corrected table passes all 6,153 universal primitive cases, with 1,618,212 symbolic rules, all 434 transition rows covered and minimum duration eight. Independent dense and sparse blank simulations reach the same exact 69,744-step frame, independently linked by the 172-step source bridge. The source binding and constant-bias calibration have been independently reviewed. Thus this is a valid corrected 217-state prototype, **not a 214/215-state result**. No exact-table Lean or Rocq endpoint is claimed.

The completeness argument for this interface is simpler than the feedback theorem. For the fields $a_1, \ldots, a_m$ of a universal refutation word, encode

$$E(r) = P(a_1, P(a_2, \ldots, P(a_m, r)\ldots)), \qquad r \geq 1.$$

Constant bias cancels every scheduler decrement, so no successor wrappers occur. Every intermediate residual is positive, and $E(r)$ executes the refutation word before reaching its unused tail. The inherited dyadic-prefix lemma implies that these successful candidates are cofinal in every residue class modulo a power of two, in particular the odd class.

For completeness, write $T_n = n(n+1)/2$ and $F(z) = T_{2z} = z(2z+1)$. Then

$$F(z) - F(w) = (z - w)(2(z + w) + 1),$$

so F permutes residues modulo 2^k. Since $P(a, b) = T_{a+b+1} - a - 1$, forcing $P(a, b) \equiv c \pmod{2^k}$ is possible by first choosing $F(z) \equiv c + a + 1$ and then imposing $b \equiv 2z - a - 1 \pmod{2^{k+1}}$. Iterate this inverse step through the fixed field list. It gives a residue class of tails modulo 2^{k+m} on which $E(r) \equiv c \pmod{2^k}$. Positive representatives are unbounded and $E(r) \geq r$, giving cofinality. Finite draining then makes odd enumeration complete. The safe startup stack three supplies soundness, conditional on the exact source, runtime and initialization obligations, without a resource-credit argument.

15 Further research questions

1. Can the corrected feedback return be physically implemented with 215 states while preserving the actual safe startup? The adjacent-pair theorem would then remove the previously open cofinality issue.
2. Can a 200-node source be encoded with a six-zero scanner without losing contextual helper sharing? A source DAG count is not a physical state bound.
3. Can successful-MP normalization or omission of its dummy zero push eliminate a physical helper rather than merely shorten the source word?
4. Can the adjacent-pair construction be generalized to blocks of k consecutive universally halting candidates for schedulers with bounded jumps k? Such a result would provide a systematic criterion for safe adaptive enumeration.
5. Can exact source CFG binding be included as an explicit checked premise in future machine-calibration endpoints, so that a correct primitive runtime cannot conceal a changed continuation target?

16 Reproduction and provenance

The packet contains exact tables, the state bijection, source leaves, fresh case records, startup traces, the return-target counterexample and owned verification scripts. Run ordinary Python with assertions enabled:

```
python3 -I -B verify.py -work /fresh/empty/path
python3 -I -B verify.py -runtime -work /fresh/full/path
```

The first command checks the listed exact finite artifacts and inherited dominated-row certificate. The second additionally reconstructs every universal primitive proof and compares its ordered case record. Neither command upgrades the rejected source to a calibrated consistency machine. Written unbounded mathematical proofs are identified separately from executable corroboration and inherited project dependencies.

The original short-transfer experiment and its declared open status are reproduced under `provenance/`. The full original gated bundle is retained in the inherited calculus; no weakened seed, omitted vacuity condition, or substituted formula-decoding assumption is used. The project snapshot and archive hashes are recorded in `provenance/UPSTREAM_PIN.json`. The directly used compiler proof is included and attributed, rather than presented as a new proof language invented in this investigation.